

Interrupt Politely

Violence isn't the answer

By Herb Sutter

Herb is a software development consultant, a software architect at Microsoft, and chair of the ISO C++ standards committee. He can be contacted at www.gotw.ca.

We want to be able to stop a running thread or task when we discover that we no longer need or want to finish it. As we saw in the last two columns, in a simple parallel search we can stop other workers once one finds a match, and when speculatively running two alternative algorithms to compute the same result we can stop the longer-running one once the first finds a result. [1,2] Stopping threads or tasks lets us reclaim their resources, including locks, and apply them to other work.

But how do you stop a thread or task you longer need or want? Table 1 summarizes the four main ways, and how they are supported on several major platforms. Let's consider them in turn.

[Click image to view at full size]

	1. Kill	2. Tell, don't take no for an answer	3. Ask politely, and accept rejection	4. Set flag politely, let it poll if it wants
Tagline	Shoot first, check invariants later	Fire him, but let him clean out his desk	Tap him on the shoulder	Send him an email
Summary	A time-honored way to randomly corrupt your state and achieve undefined behavior	Interrupt at well-defined points and allow a handler chain (but target can't refuse or stop)	Interrupt at well-defined points and allow a handler chain, but request can be ignored	Target actively checks a flag – can be manual, or provided as part of #2 or #3
Pthreads	pthread_kill pthread_cancel (async)	pthread_cancel (deferred mode)	n/a	Manual
Java	Thread.destroy Thread.stop	n/a	Thread.interrupt	Manual, or Thread.interrupted
.NET	Thread.Abort	n/a	Thread.Interrupt	Manual, or Sleep(0)
C++0x	n/a	n/a	n/a	Manual
Guidance	Avoid, almost certain to corrupt transaction(s)	OK for languages without exceptions and unwinding...	Good, conveniently automated	Good, but requires more cooperative effort (can be a plus!)

Table 1: Major cancellation/interruption options.

Option 1: (Thou Shalt Not) Kill

The first option, which is nearly always wrong, is to kill the target thread or task immediately right in the middle of whatever it happens to be doing. Th form of reckless slaughter is available in most platform APIs and frameworks, including the venerable UNIX *kill -9*, Pthreads' *pthread_kill* (or

pthread_cancel in async mode), Java *Thread.destroy* or *Thread.stop*, and .NET's *Thread.Abort*.

Every major platform has reinvented this trap because it seems like a simple idea at first, until you realize it's nearly impossible to write correct code whose execution can be abruptly killed at arbitrary and unpredictable points.

The main trouble with Option 1 is that it is an extreme measure with extreme consequences: There's rarely such a thing as killing just one thread or task. Doing that is liable not only to stop that particular work, but also to corrupt the entire process and possibly other processes. Chances are, the thread will be partway through an operation where it's taking an object or data from one valid state to another. For example, data may be partly written into a buffer; or a money-transferring task may have taken money out of one account and not yet put it into the target account. Now mix in compiler optimizers, processors, and cache subsystems that routinely transform your code and execute it out of order, and you typically have no idea just from reading the source code what memory values might be read or written, and in what orders, and therefore, no way to predict the consequences of interrupting that execution at a random point.

Killing a thread or task in the middle of doing some work usually means that we will leave behind state that has been corrupted, typically in an apparently random and unpredictable way; and/or we will lose resources the thread or task held, such as any locks it held.

Consider for a moment the specific issue of locks: If the killed thread was holding a lock, it's because it was using (and possibly changing) some data protected by that lock. Killing it in that state has two possible outcomes. First, on some platforms, the lock is released, which makes corrupted state visible to other parts of the program. Second, on other platforms, the lock is not released, which will deadlock any other parts of the program that are already waiting, or subsequently try to wait, for that same lock. Perhaps surprisingly, the second outcome is usually better, because at least it prevents the rest of the system from seeing the data that was left in a corrupt state. Of course, better still is not pulling the trigger and corrupting the data in the first place.

In short: Please, let's stop the slaughter. Option 1 is nearly always wrong because it is likely to corrupt at least the entire process, and might also corrupt other processes—including even processes on other machines if the killed thread was in the middle of performing some important I/O. Most of the time when someone tries to use *pthread_kill*, *Thread.stop*, and their ilk, the programmer is unaware of the extreme measure they're really signing up for. Be aware, and don't use it unless you really intend to take down the process or the machine without any attempt at graceful cleanup.

There are two use cases where Option 1 can be appropriate, one rare and one very rare:

- If you can prove that the target thread is doing nothing but reading memory and that it owns no resources, it may be safe to kill it.
- If you deliberately intend to terminate and restart the entire target process (not just thread) and possibly even the entire target machine, without even trying to clean up corrupted state, then killing may be appropriate. For example, in a system that uses three redundant and independent computers or processors that do not share data, when one misbehaves, it can be appropriate to kill and restart it in isolation.

Interlude: Cancellation/Interruption Points

Unlike Option 1, all of the three remaining alternatives share one vital point in common: The target thread or task can be stopped only at well-defined points in its execution, called "cancellation points" or "interruption points", which are typically when the thread is blocked doing one of the following things:

- Waiting to acquire a mutex, get a signal on a semaphore, or other synchronization.
- Joining with another thread or task.
- Sleeping.

Under Options 2, 3, and 4, these are the points at which a thread or task could be interrupted. This still imposes a burden on the author of the thread's or task's code: The code has to be ready to be interrupted at such points, and especially you have to either reestablish your invariants before you make any such calls at which you could be interrupted, or arrange for the invariants to be reestablished if you are actually interrupted. Let's see what this looks like under the remaining three Options.

Option 2: Peremptory, Don't Take No For an Answer

Option 2 is to follow the model of POSIX threads (Pthreads) deferred cancellation via *pthread_cancel*: Wait until the target thread reaches its next well-

defined cancellation point, then stop it and run the chain of cancellation handlers that the program installed (if any) which serves a similar purpose as destructor/dispose functions in modern languages. This is much better than Option 1.

The key drawback of Option 2 is that Pthreads cancellation requests cannot be ignored or caught; the target has no choice but to be stopped at its next cancellation point, and once cancellation has begun it cannot be stopped. This is a reasonable design for a language that does not have exceptions or objects with destructor/dispose functions (the cancellation handlers simulate the latter), but it is largely inappropriate for modern languages which have exception handling and know how to catch and recover from errors and continue correct execution. So Option 2 is appropriate in languages like C and Fortran if it is acceptable to force target threads to stop, but is less well suited for use with modern languages that have more sophisticated error-recovery mechanisms or your threads or tasks may legitimately want to handle the cancellation request and continue or ignore it entirely, neither of which is permitted under Option 2.

Option 3: Ask Politely

Option 3 is to follow the interruption model common to modern languages and frameworks, including Java and .NET, which spell it as *Thread.interrupt* and *Thread.Interrupt*, respectively. Like Option 2, the target thread continues to run until it reaches its next interruption point, at which point in most systems implementing Option 3 the interruption manifests as an exception thrown from the wait/join/sleep call. Then, unlike Option 2, the target thread can catch and handle the exception like any other exception, including that it has more options:

As in Pthreads, it can simply let destructors/disposers and *finally* clauses unwind the stack entirely and exit. Unlike Pthreads, the target thread can choose to unwind its stack partway until it finds a handler that catches and handles the exception, and then continue normal operations. Also unlike Pthreads, the target thread can immediately catch and ignore the exception entirely.

This is polite interruption, the state of the art in automated interruption facilities.

Option 4: Cooperate

Finally, Option 4, which you can and should use together with Option 3, is a fully cooperative model where the target thread can check to see whether someone has asked for it to interrupt work. This checking can be in between between interruption points (if you want to use both Options 3 and 4 together), or instead of interruption points (if you want to use Option 4 alone). We saw Option 4 in action in the previous two columns [1,2]: In a simple parallel search, once one worker finds an answer and records it in a shared location, the other workers can periodically check that shared location and stop their own work when they see that someone else has already found the answer.

What About Library/OS Calls?

What should you do about library calls that are not interruptible? If it doesn't cooperate, it doesn't cooperate. Don't shoot! Violence is not the answer.

What should you do if you need to call an OS (possibly kernel-mode) function that isn't interruptible? The answer is the same: If it doesn't cooperate, it doesn't cooperate. Don't shoot. Incidentally, you may have noticed a recent trend: More recent operating systems are on the road to making all calls interruptible. For example, in Windows Vista, nearly all file and I/O APIs support interruption, so that you can stop them without just waiting for them to return. This shouldn't be surprising, since we've been considering the importance of interruption in concurrent code.

Summary

Interrupt politely. Always use Options 3 and 4, which allow the thread or task to participate in the decision about whether and how it should clean up its work and/or continue on. Notify a thread of interruption requests only at well-defined predictable wait/join/sleep points, and make sure you write your code to be safe if interruption does happen at those points. Note that both Options 3 and 4 provide a strict superset of what is possible in Option 2: Anything you can code in Option 2, you can code in Option 3 or 4 as well. Avoid the peremptory Option 2 of not letting the thread participate in the decision. Even if you are running on Pthreads which does not support Option 3, you have the option of writing Option 4 yourself.

Finally, never kill a thread or task as in Option 1, unless you can prove you're truly in one of the rare cases where this questionable practice is safe and

okay to take down the whole process (or more) without any graceful cleanup at all. Most real-world attempts to kill a thread at arbitrary points are indefensible; every major threading library or environment started here, but now we know better—violence is not the answer.

Notes

[1] H. Sutter. "Going Superlinear" (*Dr. Dobbs's Journal*, March 2008).

[2] H. Sutter. "Super Linearity and the Bigger Machine" (*Dr. Dobbs's Journal*, March 2008).